

CS4055 - Mini Project Presentation

Quantum Machine Learning

Clustering using
Spherical q-Means in
MD-VRP

Anirudh Srinivasan | cs20btech11059@iith.ac.in

Dr. M.V. Panduranga Rao | mvp@cse.iith.ac.in

Agenda

- Motivation
- Clustering Algorithms
 - K-Means
 - Classical K-Means
 - q-Means
 - Spherical q-Means
- Experiments and Results
- Future Work

Introduction

MD-VRP: Multi-Depot Vehicle Routing Problem

- Depot and Customer Clustering:
 - A company may have multiple depots.
 - If customers are near depots, treat as independent VRPs.
 - If intermingled, solve MDVRP.
- MDVRP Description:
 - Assign customers to depots with vehicle fleets.
 - Vehicles start and end at the same depot.
 - Objective: Serve all customers, minimize vehicles, and travel distance.
- Formal Description:
 - Objective: Minimize fleet size, travel time sum, fulfill demand.
 - Feasibility: Routes adhere to VRP constraints, start/end at depot.
 - Formulation: Extends VRP to multiple depots, routes defined by vertices.

Clustering in MD-VRP

Algorithm 6 MD-VRP with Clustering Module

```

0: Input:  $P$  (Problem),  $V$  (Vehicles),  $C$  (Customers),  $D$  (Depots)
0: Output: Final routes  $R$ 
0: function CALCULATEDISTANCES( $N$ ) { $N$  for nodes}
0:    $d \leftarrow 0_{|N| \times |N|}$  {Initialize zero matrix for distances}
0:   for  $i \leftarrow 1$  to  $|N|$  do
0:     for  $j \leftarrow 1$  to  $|N|$  do
0:        $d[i, j] \leftarrow \sqrt{(N[i, 1] - N[j, 1])^2 + (N[i, 2] - N[j, 2])^2}$ 
0:     end for
0:   end for
0:   return  $d$ 
0: end function
0:  $N \leftarrow \text{CONCATENATE}(C, D)$ 
0:  $d \leftarrow \text{CALCULATEDISTANCES}(N)$ 
0: Clustering Module
0: Cluster  $C_i$  for each depot  $i$  based on proximity using  $d$ 
0: Compute  $|C_i|$  for each cluster
0: Initialize  $R$  {Route for each depot}
0: for  $i \leftarrow 1$  to  $|D|$  do
0:    $R_i \leftarrow [D[i]]$  {Start route at depot  $i$ }
0:    $q \leftarrow V[i, 2]$  {Initialize vehicle capacity}
0:   for  $c \in C_i$  do
0:     if  $C[c, 4] \leq q$  then {Check if customer demand fits}
0:        $R_i.append(c)$ 
0:      $q \leftarrow q - C[c, 4]$ 
0:   else
0:      $R_i.append(D[i])$  {Return to depot if capacity exceeded}
0:      $q \leftarrow V[i, 2]$  {Reset capacity}
0:   end if
0: end for
0:    $R_i.append(D[i])$  {Return to depot to complete the route}
0:    $R.append(R_i)$ 
0: end for
0: return  $R = 0$ 

```



- Depot and Customer Clustering
- Clustering is an independent module

Issues in Clustering

- Euclidean Distance Consideration:
 - Ensures clusters are physically close.
 - Prevents situations where clusters are close in angle but far apart in actual distance.
 - Optimizes routing efficiency by minimizing travel distance.
- Angle Consideration:
 - Ensures clusters are logically aligned with route directions.
 - Prevents unnecessary detours or backtracking.
 - Minimizes travel time and improves efficiency.



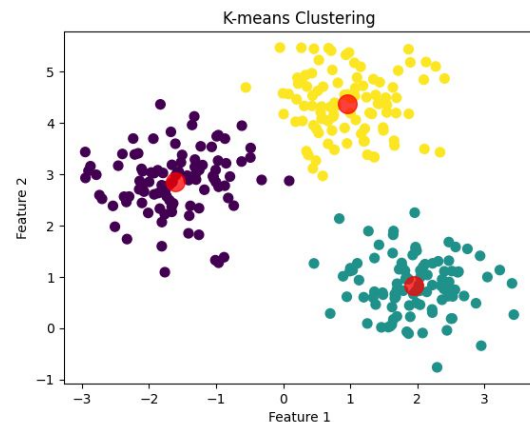
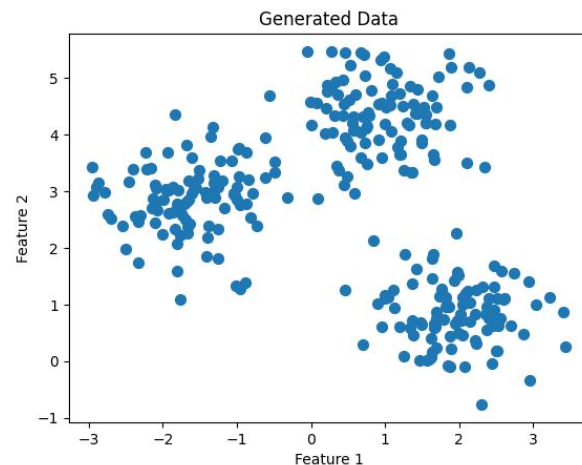
Background: Clustering Algorithms



Classical K-Means Clustering

Algorithm 1 KMeans Algorithm

- 1: **Input:** X - Set of data points, k - Number of clusters
 - 2: **Output:** C - Set of cluster centroids
 - 3: Initialize C with k randomly selected data points from X
 - 4: **repeat**
 - 5: **for** $i = 1$ to k **do**
 - 6: $S_i \leftarrow \{x \in X \mid \arg \min_{c \in C} \|x - c_i\|^2\}$
 - 7: $c_i \leftarrow \frac{1}{|S_i|} \sum_{x \in S_i} x$
 - 8: **end for**
 - 9: **until** No change in cluster assignments
 - 10: **return** C
-



The Bridge: δ -KMeans Clustering

Algorithm 2 δ -k-means algorithm

Require: Dataset V , number of clusters k , noise level δ

Ensure: Cluster centroids and labels for all data points in V

- 1: Initialize k centroids c_j^0 randomly from V
- 2: **repeat**
- 3: **for** each data point v_i in V **do**
- 4: Compute squared distances $d^2(v_i, c_j^t)$ for each centroid c_j^t
- 5: Assign v_i a label from the set $L_\delta(v_i)$ where:

$$L_\delta(v_i) = \{j : |d^2(c_i^*, v_i) - d^2(c_j, v_i)| \leq \delta\}$$

- 6: c_i^* is the closest centroid to v_i
- 7: **end for**
- 8: **for** each cluster $j = 1$ to k **do**
- 9: Calculate new centroids c_j^{t+1} :

$$c_j^{t+1} = \frac{1}{|C_j^t|} \sum_{v_i \in C_j^t} v_i$$

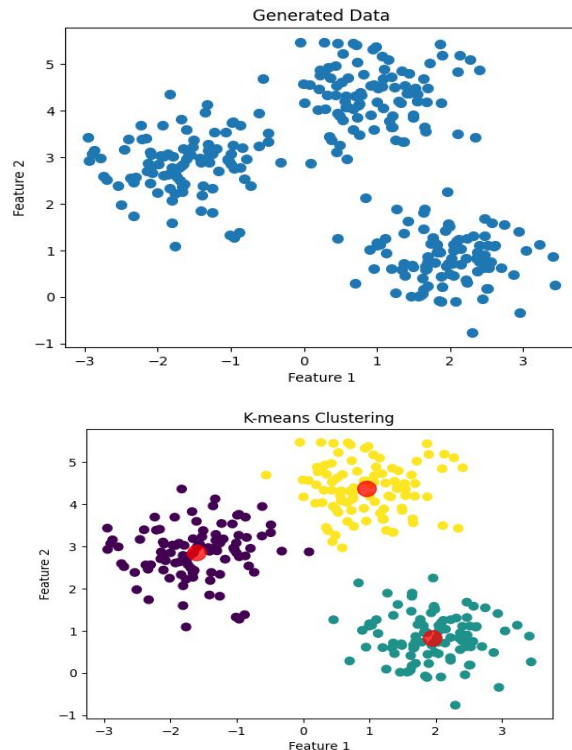
- 10: Add noise:

$$\text{Induce noise } \frac{\delta}{2} \text{ to } c_j^{t+1}$$

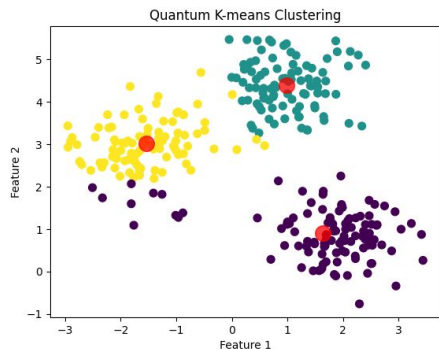
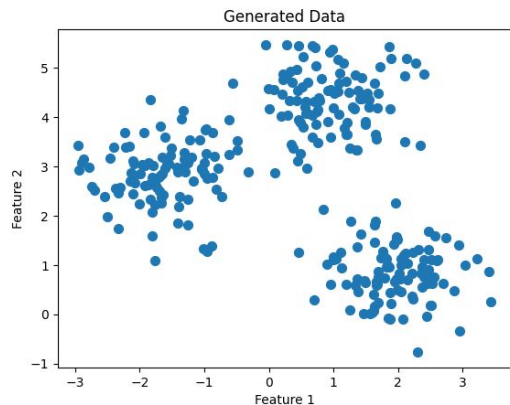
- 11: Ensure c_j^{t+1} satisfies:

$$d\left(c_j^{t+1}, \frac{1}{|C_j^{t+1}|} \sum_{v_i \in C_j^{t+1}} v_i\right) < \frac{\delta}{2}$$

- 12: **end for**
 - 13: $t \leftarrow t + 1$
 - 14: **until** convergence criterion is met
-



Quantum K-Means Clustering



Algorithm 3 q-means Algorithm

Require: Quantum dataset V , number of clusters k , error bounds ϵ_3, ϵ_4

Ensure: Quantum centroids and labels for data points in V

- 1: Initialize k quantum centroids c_j^0 randomly
- 2: **repeat**
- 3: **Centroid Distance Estimation:**
- 4: **for** each data point v_i in V **do**
- 5: Prepare quantum state $\frac{1}{\sqrt{N}} \sum_{i=1}^N |i\rangle \otimes_{j \in [k]} \left(|j\rangle \left| d^2(v_i, c_j^t) \right\rangle \right)$
- 6: Compute distances $d^2(v_i, c_j^t)$ using the superposition of data points and centroids
- 7: **end for**
- 8: **Cluster Assignment:**
- 9: **for** each data point v_i in V **do**
- 10: Assign v_i to cluster by finding $\arg \min_j d(v_i, c_j^t)$ using a $U_{\min}$ circuit
- 11: Prepare state $|\psi^t\rangle = \frac{1}{\sqrt{N}} \sum_{i=1}^N |i\rangle |\ell^t(v_i)\rangle$
- 12: **end for**
- 13: **Centroid State Creation:**
- 14: Update and store centroid states:

$$|\psi^t\rangle = \sum_{j=1}^k \sqrt{\frac{|C_j|}{N}} \left(\frac{1}{\sqrt{|C_j|}} \sum_{i \in C_j} |i\rangle \right) |j\rangle$$

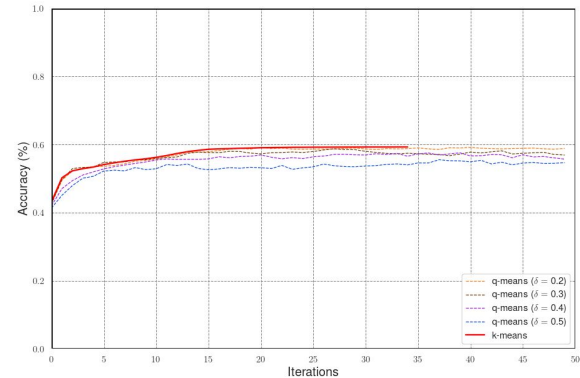
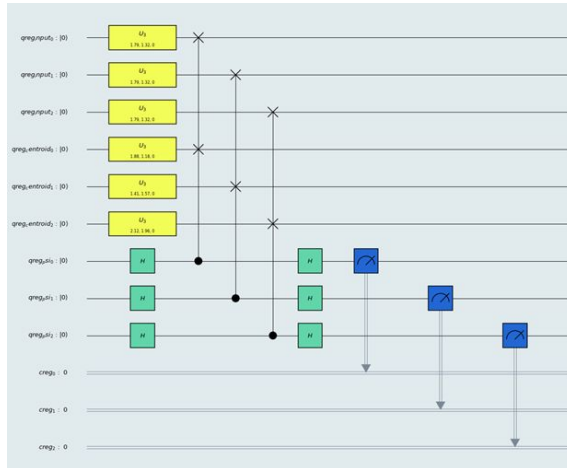
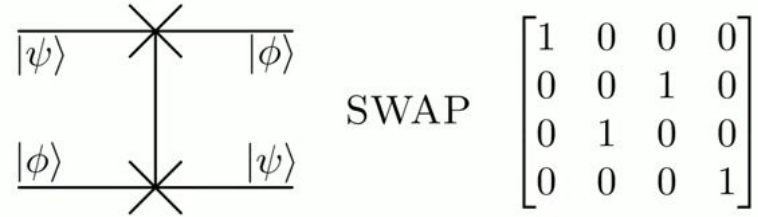
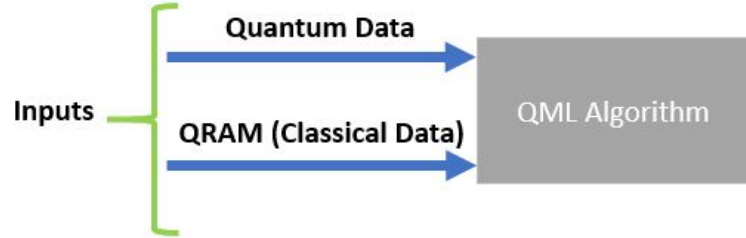
- 15: **Centroid Update:**
- 16: **for** each cluster $j = 1$ to k **do**
- 17: Convert quantum description of centroids to classical description:

$$\|c_j^{t+1}\rangle - \|\overline{c_j^{t+1}}\rangle\| < \epsilon_4$$

$$\|c_j\| - \|\overline{c_j}\| \leq \epsilon_3 \|c_j\|$$

- 18: **end for**
 - 19: $t \leftarrow t + 1$
 - 20: **until** convergence or maximum iterations reached
-

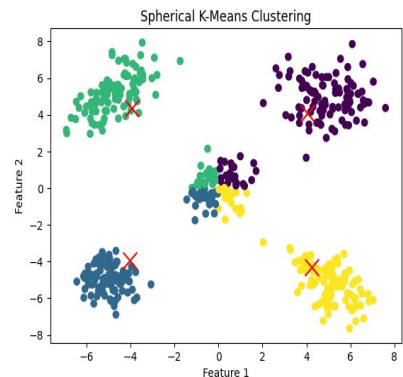
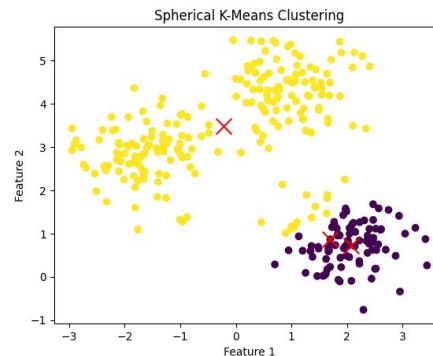
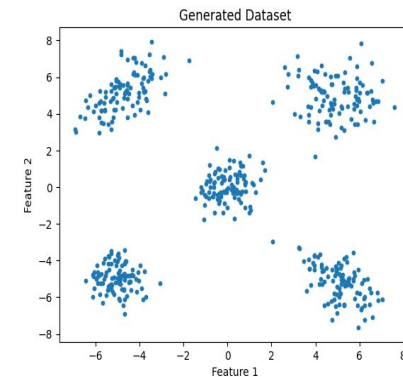
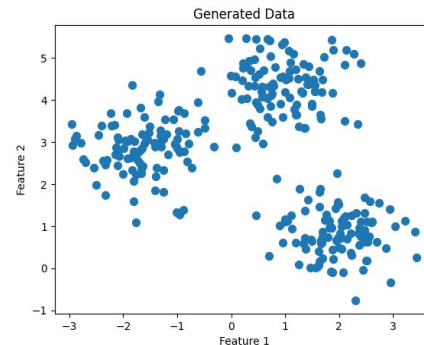
A DeepDive into the Algorithm!



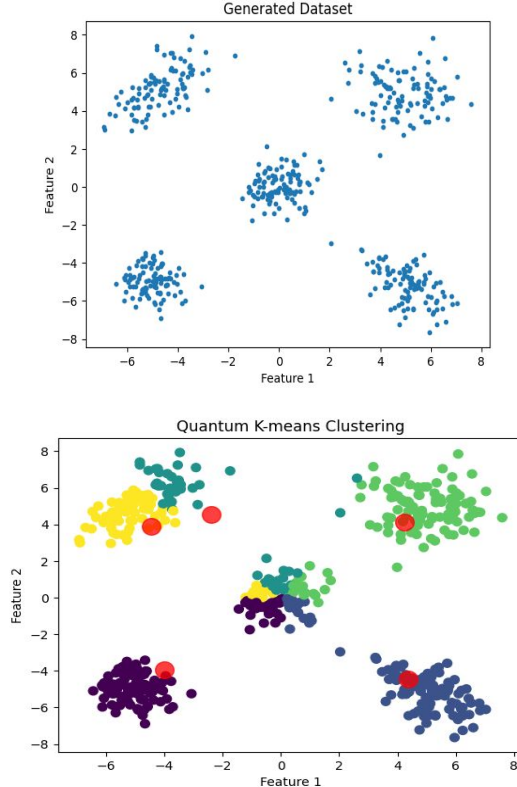
Classical Spherical K-Means

Algorithm 4 Spherical K-Means Algorithm

- 1: **Input:** X - Set of data points, k - Number of clusters
 - 2: **Output:** C - Set of cluster centroids
 - 3: Normalize each data point in X to have unit norm
 - 4: Initialize C with k randomly selected normalized data points from X
 - 5: **repeat**
 - 6: **for** $i = 1$ to k **do**
 - 7: $S_i \leftarrow \{x \in X \mid \arg \min_{c \in C} (1 - \langle x, c_i \rangle)\}$
 - 8: $c_i \leftarrow \frac{1}{|S_i|} \sum_{x \in S_i} x$
 - 9: Normalize c_i to have unit norm
 - 10: **end for**
 - 11: **until** No change in cluster assignments
 - 12: **return** C
-



Spherical q-Means: The Quantum Analogue!



Algorithm 5 Spherical q-Means Algorithm

Require: Quantum dataset V , number of clusters k , error bounds ϵ_3, ϵ_4

Ensure: Quantum centroids and labels for data points in V , all normalized to unit length

- 1: Normalize each data point v_i in V to lie on the unit sphere
- 2: Initialize k quantum centroids c_j^0 randomly and normalize to unit length
- 3: **repeat**
- 4: **Centroid Distance Estimation:**
- 5: **for** each data point v_i in V **do**
- 6: Prepare quantum state $\frac{1}{\sqrt{N}} \sum_{i=1}^N |i\rangle \otimes_{j \in [k]} \left(|j\rangle \frac{1 - \langle v_i | c_j \rangle}{2} \right)$
- 7: Compute distances based on the dot product $\langle v_i | c_j \rangle$ indicating cosine similarity
- 8: **end for**
- 9: **Cluster Assignment:**
- 10: **for** each data point v_i in V **do**
- 11: Assign v_i to cluster by finding $\arg \max_j \langle v_i, c_j^t \rangle$ using a $U_{\max}$ circuit
- 12: Prepare state $|\psi^t\rangle = \frac{1}{\sqrt{N}} \sum_{i=1}^N |i\rangle |c^t(v_i)\rangle$
- 13: **end for**
- 14: **Centroid State Creation:**
- 15: Update and store centroid states:

$$|\psi^t\rangle = \sum_{j=1}^k \sqrt{\frac{|C_j|}{N}} \left(\frac{1}{\sqrt{|C_j|}} \sum_{i \in C_j} |i\rangle \right) |j\rangle$$

- 16: Normalize each centroid c_j^{t+1}
- 17: **Centroid Update:**
- 18: **for** each cluster $j = 1$ to k **do**
- 19: Convert quantum description of centroids to classical description, ensuring they remain unit vectors:

$$\|c_j^{t+1}\rangle - \overline{\|c_j^{t+1}\rangle}\| < \epsilon_4$$

$$\|c_j\| - \overline{\|c_j\|} \leq \epsilon_3 \|c_j\|$$

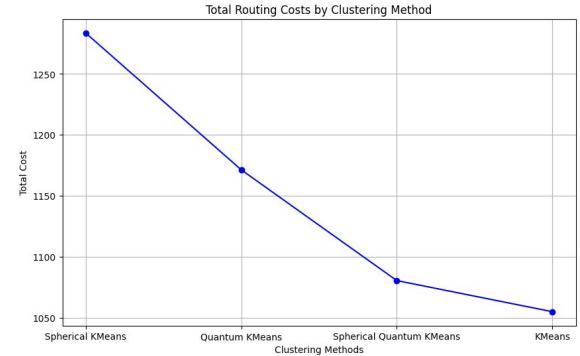
- 20: **end for**
 - 21: $t \leftarrow t + 1$
 - 22: **until** convergence or maximum iterations reached
-

Results with VRP Datasets

Results

Table 1: Clustering Performance Metrics

Clustering Method	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Index
KMeans	0.378	0.787	83.23
Spherical KMeans	0.221	0.762	46.89
qMeans	0.200	0.889	54.01
Spherical qMeans	0.295	0.742	62.62



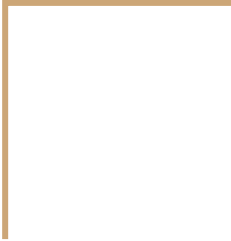
- Tested across 13 MD-VRP Benchmark Datasets (avg. values are reported)
- KMeans and Spherical qMeans consistently perform better than the other methods

Future Work

- Hybrid Distance Metric (Euclidean Distance + Cosine Distance)
 - Provides a holistic view of cluster assignments.
 - Optimizes vehicle routing solutions by considering both directionality and physical proximity.
 - Leads to more efficient delivery routes and reduced operational costs.
- Tweak the quantum circuits in qMeans algorithm to have better control
- New Evaluation Metric tailored to VRP

References

1. [Spherical k-Means Clustering](#)
2. [K-Cosine-Means Clustering Algorithm](#)
3. [q-means: A quantum algorithm for unsupervised machine learning](#)
4. [Discriminating Quantum States with Quantum Machine Learning](#)
5. [A new geometric shape-based genetic clustering algorithm for the multi-depot vehicle routing problem](#)
6. <http://www.vrp-rep.org/datasets.html>



Thank You

Any questions?

